BCS702 - Parallel Programming

Unit 3 (MPI) and Unit 4 (OpenMP)

Question Bank with Model Answers - 10 Marks Each (VTU Exam Format)

Q. No.	Question
U3 - Q1	Explain collective communication in MPI programs.
U3 - Q2	Explain tree based communication in MPI programs.
U3 - Q3	Explain the working of MPI_Bcast() and MPI_Scatter().
U3 - Q4	Explain MPI_Gather() and MPI_Reduce() with suitable examples.
U3 - Q5	Differentiate point-to-point communication and collective communication in MPI.
U3 - Q6	Write an MPI program to calculate the sum of N numbers.
U3 - Q7	Explain the collective functions of MPI.
U4 - Q1	Differentiate between critical and atomic directive.
U4 - Q2	What are OpenMP directives? Explain them with examples.
U4 - Q3	Explain different types of scheduling in OpenMP.
U4 - Q4	Explain parallel loops using the parallel for directive. Write a coded example.

Writing tip: write the **question number**, give a short **introduction**, then the **headings** shown here, draw the **diagram** where given, write **syntax / code** neatly, and finish with a **conclusion**. Underline the **bold keywords** in your answer.

UNIT 3 - Distributed Memory Programming with MPI

Q. 1 Explain collective communication in MPI programs. [10 Marks]

Introduction

In MPI, a **communicator** is a collection of processes that can send messages to each other (default: `MPI_COMM_WORLD`). Communication functions that involve **all the processes in a communicator** are called **collective communications**. Functions like `MPI_Send` and `MPI_Recv`, which involve only two processes, are called **point-to-point communications**.

Need for collective communication

In the trapezoidal rule program, every process with rank > 0 sends its result to process 0, which receives and adds them all. Process 0 does almost all the work of the global sum while the others do almost nothing. Collective functions let MPI share this work (using a tree structure) and move the burden of optimisation from the programmer to the MPI implementer.

Common collective functions

Function	Purpose
`MPI_Reduce`	Combines values of all processes (sum, max, ...) and stores result on one process
`MPI_Allreduce`	Same as reduce, result stored on all processes
`MPI_Bcast`	Sends data from one process to all processes
`MPI_Scatter`	Distributes pieces of an array from one process to all
`MPI_Gather`	Collects pieces from all processes on one process
`MPI_Allgather`	Gather, with the full result stored on every process
`MPI_Barrier`	Synchronises all the processes

MPI_Reduce

```
int MPI_Reduce(void* input_data_p, void* output_data_p, int count,
               MPI_Datatype datatype, MPI_Op operator,
               int dest_process, MPI_Comm comm);
```

Operators: `MPI_MAX`, `MPI_MIN`, `MPI_SUM`, `MPI_PROD`, `MPI_LAND`, `MPI_BAND`, `MPI_LOR`, `MPI_BOR`, `MPI_LXOR`, `MPI_BXOR`, `MPI_MAXLOC`, `MPI_MINLOC`.

Example: `MPI_Reduce(&local_int, &total_int, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);` With `count` > 1 it works on arrays, e.g. adding N-dimensional vectors.

Rules of collective communication

1. All processes in the communicator must call the **same** collective function.
2. Arguments passed by all processes must be **compatible** (e.g. same `dest_process`).
3. `output_data_p` is used only on `dest_process`, but every process must pass some argument (can be `NULL`).
4. **No tags** are used. Calls are matched only by **communicator and order of calls**.
5. **Argument aliasing is illegal**: `MPI_Reduce(&x, &x, ...)` gives unpredictable results.

Conclusion

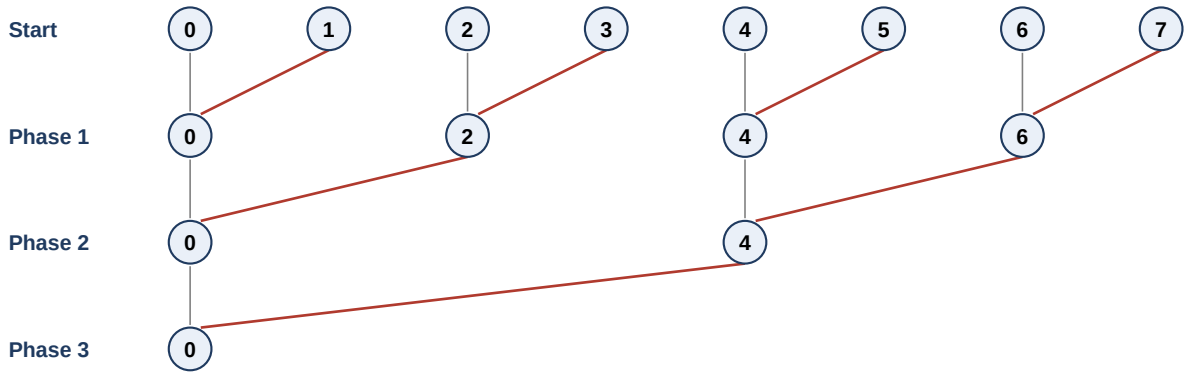
Collective communication involves every process of a communicator, shares work equally, is simple to code and is optimised by MPI.

Q. 2 Explain tree based communication in MPI programs. [10 Marks]

Introduction

In a simple global sum, process 0 receives `comm_sz - 1` values and adds them one by one, so it does all the work. To share the work, a **binary tree structure** is used. This is called **tree-structured communication**.

Working (8 processes, 0 to 7)



Tree-structured global sum (8 processes): process 0 needs only 3 receives

1. **Phase 1:** 1, 3, 5, 7 send their values to 0, 2, 4, 6. The receivers add the values.
2. **Phase 2:** 2 and 6 send to 0 and 4. Processes 0 and 4 add.
3. **Phase 3:** 4 sends to 0. Process 0 adds and holds the **global sum**.

Advantages

- Original scheme: process 0 does **7 receives and 7 adds**. Tree scheme: only **3**; other processes do at most two.
- Much of the work is done **concurrently** (receives and adds by 0, 2, 4, 6 in phase 1 happen simultaneously). Total time is reduced by **more than 50%**.
- For `comm_sz = 1024`: original needs 1023 receives; the tree needs only **10**, an improvement by more than a **factor of 100**.

Alternative tree structure

Other pairings are possible, e.g. Phase 1: (0,4), (1,5), (2,6), (3,7); Phase 2: (0,2), (1,3); Phase 3: (0,1). The best one may depend on the tree size and on the system, so it is unreasonable to code and test every alternative.

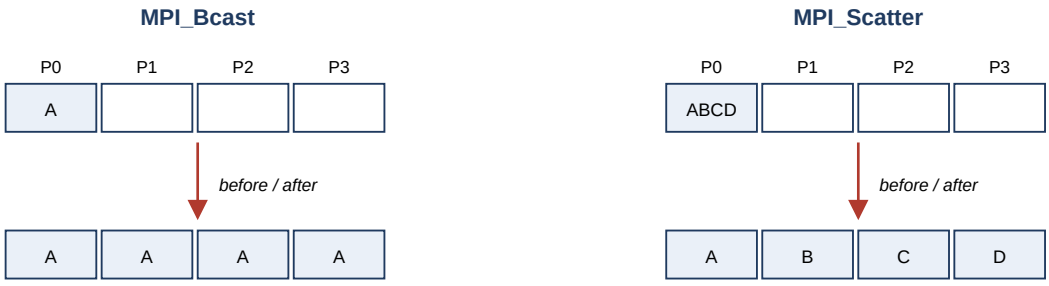
Use of tree-structured communication in MPI

- **MPI_Reduce:** MPI requires implementations to provide an optimised global sum, so the programmer simply calls `MPI_Reduce`.
- **MPI_Bcast:** a tree-structured broadcast is the **reverse** of the tree-structured global sum.
- **MPI_Allreduce:** the branches of the tree are reversed to distribute the result, or processes exchange partial results in a **butterfly** structure.

Conclusion

Tree-structured communication cuts the work of process 0 from `comm_sz - 1` steps to about `log2(comm_sz)` steps and is the basis of `MPI_Reduce`, `MPI_Bcast` and `MPI_Allreduce`.

Q. 3 Explain the working of MPI_Bcast() and MPI_Scatter(). [10 Marks]



A. MPI_Bcast()

Broadcast: a collective communication in which data belonging to a **single process is sent to all processes** in the communicator.

```
int MPI_Bcast(void* data_p /*in/out*/, int count, MPI_Datatype datatype,
              int source_proc, MPI_Comm comm);
```

- The process with rank `source_proc` sends the contents of `data_p` to all processes in `comm`.
- Implemented with a **tree-structured** communication (reverse of tree sum).
- `data_p` is an **input** on the source process and an **output** on the others, hence `in/out`.

Example: only process 0 reads `a`, `b`, `n`; then broadcasts them.

```
if (my_rank == 0) { printf("Enter a, b, and n\n"); scanf("%lf %lf %d", a_p, b_p, n_p); }
MPI_Bcast(a_p, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
MPI_Bcast(b_p, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
MPI_Bcast(n_p, 1, MPI_INT,    0, MPI_COMM_WORLD);
```

B. MPI_Scatter()

Need: if process 0 broadcasts a vector of 10,000 elements, every process must allocate 10,000 elements though it needs only its own block. `MPI_Scatter` sends only the needed block to each process.

```
int MPI_Scatter(void* send_buf_p, int send_count, MPI_Datatype send_type,
                void* recv_buf_p, int recv_count, MPI_Datatype recv_type,
                int src_proc, MPI_Comm comm);
```

- The data in `send_buf_p` is divided into `comm_sz` pieces: the **first piece goes to process 0, second to process 1**, and so on.
- With block distribution, `local_n = n / comm_sz`; each process receives `local_n` elements (`recv_count = local_n`).
- `send_count` is the amount of data sent to **each** process (not the total), so it is also `local_n`.
- Suitable only for a **block distribution**, and `n` must be **evenly divisible** by `comm_sz`.

```
MPI_Scatter(a, local_n, MPI_DOUBLE, local_a, local_n, MPI_DOUBLE, 0, comm);
```

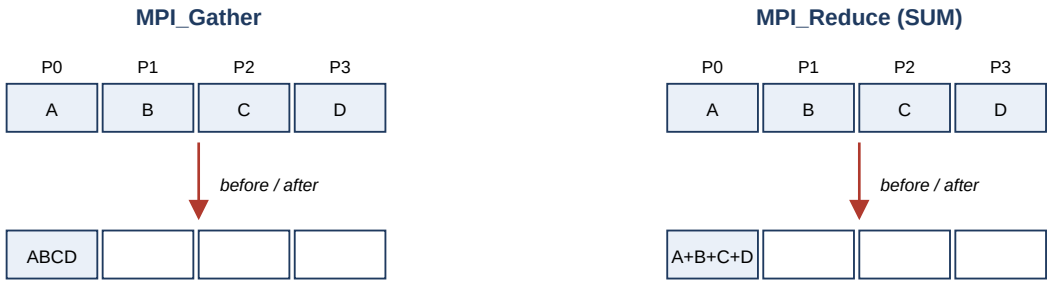
Data distributions (12 elements, 3 processes)

Process	Block	Cyclic	Block-cyclic (size 2)
0	0 1 2 3	0 3 6 9	0 1 6 7
1	4 5 6 7	1 4 7 10	2 3 8 9
2	8 9 10 11	2 5 8 11	4 5 10 11

Difference

`MPI_Bcast` sends the **same data** to all processes; `MPI_Scatter` sends a **different piece** to each process.

Q. 4 Explain MPI_Gather() and MPI_Reduce() with suitable examples. [10 Marks]



A. MPI_Gather()

The **inverse of MPI_Scatter**. It collects the pieces held by all processes and stores them together on one process.

```
int MPI_Gather(void* send_buf_p, int send_count, MPI_Datatype send_type,
               void* recv_buf_p, int recv_count, MPI_Datatype recv_type,
               int dest_proc, MPI_Comm comm);
```

- Data of process 0 is stored in the **first block** of `recv_buf_p`, process 1 in the second block, and so on.
- `recv_count` is the number of items received **from each process**, not the total.
- Works correctly only for a **block distribution** with equal block sizes.

Example: printing a distributed vector

```
void Print_vector(double local_b[], int local_n, int n, char title[],
                  int my_rank, MPI_Comm comm) {
    double* b = NULL;  int i;
    if (my_rank == 0) {
        b = malloc(n * sizeof(double));
        MPI_Gather(local_b, local_n, MPI_DOUBLE, b, local_n, MPI_DOUBLE, 0, comm);
        printf("%s\n", title);
        for (i = 0; i < n; i++) printf("%f ", b[i]);
        printf("\n");  free(b);
    } else
        MPI_Gather(local_b, local_n, MPI_DOUBLE, b, local_n, MPI_DOUBLE, 0, comm);
}
```

B. MPI_Reduce()

Combines values from all processes using an **operator** and stores the result on the **destination process**.

```
int MPI_Reduce(void* input_data_p, void* output_data_p, int count,
               MPI_Datatype datatype, MPI_Op operator, int dest_process, MPI_Comm comm);
```

- Operators: `MPI_MAX`, `MPI_MIN`, `MPI_SUM`, `MPI_PROD`, `MPI_LAND`, `MPI_BAND`, `MPI_LOR`, `MPI_BOR`, `MPI_LXOR`, `MPI_BXOR`, `MPI_MAXLOC`, `MPI_MINLOC`.
- Internally uses tree-structured communication. The result is available only on `dest_process`.
- Input and output buffers must be different (no aliasing).

Example 1: global sum in trapezoidal rule: `MPI_Reduce(&local_int, &total_int, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);`

Example 2: adding vectors: `MPI_Reduce(local_x, sum, N, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);`

Gather vs Reduce

MPI_Gather	MPI_Reduce
Collects (concatenates) all pieces	Combines data into one value using an operator
No operator needed	Needs an operator such as `MPI_SUM`
Result grows with number of processes	Result has size `count`

Related: `MPI_Allreduce` and `MPI_Allgather` store the result on **all** processes.

Q. 5 Differentiate point-to-point communication and collective communication in MPI. [10 Marks]

Introduction

Point-to-point communication (`MPI_Send`, `MPI_Recv`) involves exactly two processes. **Collective communication** (`MPI_Reduce`, `MPI_Bcast`, `MPI_Scatter`, `MPI_Gather`, ...) involves all processes in a communicator.

Differences

Point	Point-to-point	Collective
Processes involved	Exactly two (sender and receiver)	All processes in the communicator
Examples	`MPI_Send`, `MPI_Recv`	`MPI_Reduce`, `MPI_Allreduce`, `MPI_Bcast`, `MPI_Scatter`, `MPI_Gather`, `MPI_Allgather`, `MPI_Barrier`
Function called	Different functions at the two ends (send / receive)	**Same** function called by all processes
Matching	By **tags** and communicator	By **communicator and order of calls**; **no tags**
Arguments	Describe own buffer, source or destination	Must be **compatible** on all processes
Output argument	Used by the receiver	Used only on destination, but all must pass one (can be `NULL`)
Blocking	`MPI_Recv` blocks; `MPI_Send` may block or buffer	Return when the process's part of the operation is complete
Work sharing	Programmer codes loops of send/recv; process 0 often does most work	MPI uses **tree / butterfly** to share work
Programming effort	More code, programmer optimises	One call, optimised by MPI implementer
Aliasing	Not applicable	Same input and output buffer is illegal
Wrong usage	Lost message / blocking	Program likely hangs or crashes

Example

Global sum using point-to-point (process 0 receives from everyone):

```
if (my_rank != 0)
    MPI_Send(&local_int, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD);
else {
    total_int = local_int;
    for (source = 1; source < comm_sz; source++) {
        MPI_Recv(&local_int, 1, MPI_DOUBLE, source, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        total_int += local_int;
    }
}
```

Same sum using collective communication:

```
MPI_Reduce(&local_int, &total_int, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
```

Conclusion

Point-to-point is a two-process, tag-matched exchange; collective is an all-process operation matched by order, simpler and usually faster.

Q. 6 Write an MPI program to calculate the sum of N numbers. [10 Marks]

Algorithm

1. Initialise MPI; get `my_rank` and `comm_sz`.
2. Process 0 reads n; `MPI_Bcast` sends n to all. Compute `local_n = n / comm_sz`.
3. Process 0 reads the n numbers; `MPI_Scatter` gives each process `local_n` numbers.
4. Each process adds its numbers into `local_sum`.
5. `MPI_Reduce` with `MPI_SUM` adds all local sums into `total_sum` on process 0.
6. Process 0 prints the result; `MPI_Finalize`.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(void) {
    int     my_rank, comm_sz, n = 0, local_n, i;
    double *a = NULL, *local_a, local_sum = 0.0, total_sum = 0.0;

    MPI_Init(NULL, NULL);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);

    if (my_rank == 0) {                     /* only process 0 reads input */
        printf("Enter the value of N\n");
        scanf("%d", &n);
    }
    MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);

    local_n = n / comm_sz;                  /* numbers per process */
    local_a = malloc(local_n * sizeof(double));

    if (my_rank == 0) {
        a = malloc(n * sizeof(double));
        printf("Enter %d numbers\n", n);
        for (i = 0; i < n; i++) scanf("%lf", &a[i]);
    }
    MPI_Scatter(a, local_n, MPI_DOUBLE, local_a, local_n, MPI_DOUBLE,
                0, MPI_COMM_WORLD);

    for (i = 0; i < local_n; i++)           /* local sum */
        local_sum += local_a[i];

    MPI_Reduce(&local_sum, &total_sum, 1, MPI_DOUBLE, MPI_SUM, 0,
               MPI_COMM_WORLD);             /* global sum */

    if (my_rank == 0) {
        printf("Sum of %d numbers = %f\n", n, total_sum);
        free(a);
    }
    free(local_a);
    MPI_Finalize();
    return 0;
}
```

Compile and run

```
$ mpicc -g -Wall -o mpi_sum mpi_sum.c
$ mpiexec -n 4 ./mpi_sum
```

Sample (4 processes, N = 8, numbers 1 to 8)

Process	Numbers received	Local sum
0	1, 2	3
1	3, 4	7
2	5, 6	11
3	7, 8	15

`MPI_Reduce` adds 3 + 7 + 11 + 15, and process 0 prints **36**.

Key points

- **SPMD:** one program; `if (my_rank == 0)` makes processes do different work.
- `MPI_Bcast` shares `n`; `MPI_Scatter` sends only the needed block; `send_count` = amount sent to **each** process.
- `MPI_Reduce` uses a tree-structured sum; input and output buffers are different (no aliasing).
- Assumes `n` is evenly divisible by `comm_sz`.

Q. 7 Explain the collective functions of MPI. [10 Marks]

Introduction

Collective functions involve **all processes in a communicator**. They use no tags, are matched by communicator and order of calls, must be called by every process, and are implemented with tree-structured or butterfly communication.

1. MPI_Reduce

Combines a value (or array) from each process using an operator and stores the result on **one destination process**.

```
int MPI_Reduce(void* input_data_p, void* output_data_p, int count,
               MPI_Datatype datatype, MPI_Op operator, int dest_process, MPI_Comm comm);
```

Operators: `MPI_MAX`, `MPI_MIN`, `MPI_SUM`, `MPI_PROD`, `MPI_LAND`, `MPI_BAND`, `MPI_LOR`, `MPI_BOR`, `MPI_LXOR`, `MPI_BXOR`, `MPI_MAXLOC`, `MPI_MINLOC`.

2. MPI_Allreduce

Same as `MPI_Reduce`, but the result is stored on **all** processes (no `dest_process` argument). Uses reverse tree or **butterfly**.

```
int MPI_Allreduce(void* input_data_p, void* output_data_p, int count,
                  MPI_Datatype datatype, MPI_Op operator, MPI_Comm comm);
```

3. MPI_Bcast

Sends data from **one process to all** processes (tree-structured broadcast).

```
int MPI_Bcast(void* data_p, int count, MPI_Datatype datatype, int source_proc, MPI_Comm comm);
```

4. MPI_Scatter

Distributes the elements of an array: first `n/p` to process 0, next `n/p` to process 1, etc. (block distribution).

```
int MPI_Scatter(void* send_buf_p, int send_count, MPI_Datatype send_type,
                void* recv_buf_p, int recv_count, MPI_Datatype recv_type,
                int src_proc, MPI_Comm comm);
```

5. MPI_Gather

Inverse of scatter. Collects the sub-arrays of all processes on a designated process, process 0's data first, then process 1's, etc.

```
int MPI_Gather(void* send_buf_p, int send_count, MPI_Datatype send_type,
               void* recv_buf_p, int recv_count, MPI_Datatype recv_type,
               int dest_proc, MPI_Comm comm);
```

6. MPI_Allgather

Like `MPI_Gather`, but the concatenated data is stored on **every** process (gather + broadcast done better using a butterfly). Used in matrix-vector multiplication so every process gets the whole vector `x`.

```
int MPI_Allgather(void* send_buf_p, int send_count, MPI_Datatype send_type,
                  void* recv_buf_p, int recv_count, MPI_Datatype recv_type, MPI_Comm comm);
```

7. MPI_Barrier

Synchronises the processes. No process returns from `MPI_Barrier` until all processes in the communicator have started the call.

Summary

Function	Data movement	Result on
`MPI_Reduce`	All to one, with operation	One process
`MPI_Allreduce`	All to all, with operation	All processes
`MPI_Bcast`	One to all (same data)	All processes
`MPI_Scatter`	One to all (different pieces)	Each gets one piece
`MPI_Gather`	All to one (concatenate)	One process
`MPI_Allgather`	All to all (concatenate)	All processes
`MPI_Barrier`	Synchronisation only	No data

UNIT 4 - Shared-Memory Programming with OpenMP

Q. 1 Differentiate between critical and atomic directive. [10 Marks]

Introduction

When two or more threads update a shared variable simultaneously, a **race condition** occurs. Example: `global_result = 0`, thread 0 has `my_result = 1`, thread 1 has `my_result = 2`; `global_result += my_result` may produce 1 or 2 instead of 3. The code that updates the shared resource is a **critical section**: it is executed by multiple threads, but only one thread at a time may update the shared resource. OpenMP provides `critical` and `atomic` directives (and locks) for **mutual exclusion**.

critical directive

```
#  pragma omp critical
   global_result += my_result;
```

- Only **one thread at a time** executes the following structured block.
- Can protect **any structured block** (several statements in `{ }`, function calls).
- All unnamed `critical` blocks form one composite critical section. A **name** can be given, `#pragma omp critical(name)`; blocks with different names can run simultaneously (names are fixed at compile time).

atomic directive

```
#  pragma omp atomic
   done_sending++;
```

- Protects only a **single C assignment statement** of the form: `x <op>= <expression>; x++; ++x; x--; --x;`
- `<op>` is one of `+ * - / & ^ | << >>`; `<expression>` must **not reference x**.
- Only the **load and store of x** are protected. In `x += y++;` the update of `y` may not be protected.
- Uses the processor's special **load-modify-store instruction**, so it can be the **fastest** method.
- The standard allows an implementation to treat **all** `atomic` directives as one critical section (`x++` and `y++` may not run simultaneously). Statements modifying the **same** variable are always mutually exclusive.

Differences

Point	critical	atomic
Syntax	`#pragma omp critical` or `critical(name)`	`#pragma omp atomic`
Protects	Any structured block	Only a single assignment statement of allowed form
Statements allowed	Any statements, function calls, many lines	`x <op>= expr`, `x++`, `++x`, `x--`, `--x`
Implementation	General mutual-exclusion mechanism	Special hardware load-modify-store instruction
Speed	Slower in general	Potentially the fastest
Naming	Can be named	Cannot be named
Typical use	Enqueue / dequeue of a message queue	Counter update like `done_sending++`

Which to use?

- If the critical section is an assignment of the required form, use **atomic**.
- Otherwise use **critical** (named or unnamed); it is easy to use and performs close to locks.
- Use **locks** when mutual exclusion is needed for a **data structure** rather than a block of code.

Caveats

1. **Do not mix** different types of mutual exclusion for the same critical section (an `atomic` and a `critical` updating `x` do not exclude each other).
2. There is **no guarantee of fairness**: a thread may be blocked forever.
3. **Nesting** is dangerous: a `critical` inside a function called from another `critical` block **deadlocks**. Different names do not help if sections are entered in different orders.

Conclusion

`critical` is general and flexible; `atomic` is restricted to simple updates but faster. Both prevent race conditions.

Q. 2 What are OpenMP directives? Explain them with examples. [10 Marks]

Introduction

OpenMP is an API for **shared-memory MIMD** programming. It is a **directives-based** API: in C/C++ the directives are preprocessor instructions called **pragmas**. A compiler that does not support them ignores them and the code runs sequentially. OpenMP also has a library of functions and macros (header `omp.h`) and was designed to **incrementally parallelize** serial programs.

Format

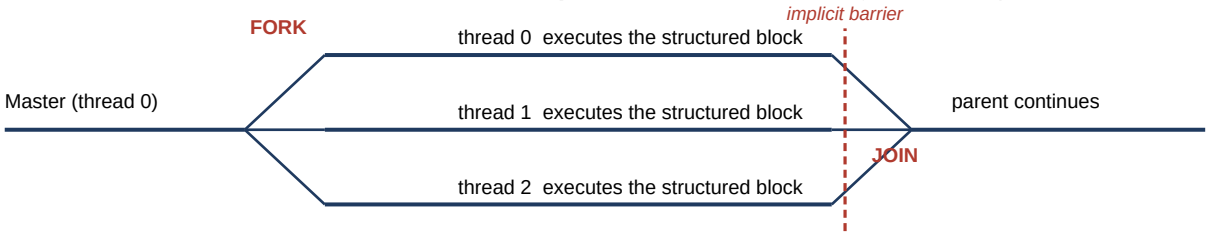
```
#  pragma omp <directive-name> [clause, clause, ...]
```

- A **clause** modifies a directive, e.g. `num_threads`, `reduction`, `private`, `schedule`.
- Directives apply to a **structured block**: one entry point and one exit point.
- A long pragma is continued on the next line with a backslash `\`.
- Compile: `gcc -g -Wall -fopenmp -o omp_hello omp_hello.c`; run: `./omp_hello 4`.

Main directives

1. parallel

Forks a **team of threads** to execute the structured block. Original thread (**master**/parent) continues and `thread_count - 1` **child** threads are started. At the end there is an **implicit barrier**; child threads join and the parent continues.




```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
void Hello(void);

int main(int argc, char* argv[]) {
    int thread_count = strtol(argv[1], NULL, 10);
#   pragma omp parallel num_threads(thread_count)
    Hello();
    return 0;
}
void Hello(void) {
    int my_rank = omp_get_thread_num();
    int thread_count = omp_get_num_threads();
    printf("Hello from thread %d of %d\n", my_rank, thread_count);
}
```

2. parallel for

Forks a team and **divides the iterations** of the following `for` loop among the threads.

```
#  pragma omp parallel for num_threads(thread_count) reduction(+: approx)
   for (i = 1; i <= n-1; i++)
       approx += f(a + i*h);
```

3. for

Parallelizes a loop using the threads **already forked** by an enclosing `parallel` block (no new threads). Has an implicit barrier at the end.

```
#  pragma omp parallel num_threads(thread_count)
   for (phase = 0; phase < n; phase++) {
#      pragma omp for
       for (i = 1; i < n; i += 2) { /* compare and swap */ }
   }
```

4. critical 5. atomic

```
#  pragma omp critical
   global_result += my_result;

#  pragma omp atomic
   done_sending++;
```

6. barrier

Every thread blocks until **all threads in the team** reach the barrier. `# pragma omp barrier`

7. task, single, taskwait (Tasking)

- `task`: generates a task that the run-time schedules; used for recursive and dynamic problems.
- `single`: tasks are launched from only one thread of the team (inside a `parallel` region).
- `taskwait`: a task waits for its sub-tasks to complete (barrier for tasks).

```
#  pragma omp parallel
#  pragma omp single
   {
#      pragma omp task shared(i)
       i = fib(n - 1);
#      pragma omp task shared(j)
       j = fib(n - 2);
#      pragma omp taskwait
       fibs[n] = i + j;
   }
```

Important clauses

Clause	Meaning
`num_threads(n)`	Number of threads to start
`reduction(op: var)`	Private copy per thread (initialised to identity value); copies combined into shared variable at end
`private(var)`	Each thread has its own copy; value unspecified at start and end of block
`shared(var)`	Variable shared by all threads
`default(none)`	Programmer must specify scope of every variable
`schedule(type, chunk)`	How loop iterations are assigned to threads

Scope: variables declared **before** a parallel block are **shared**; variables declared **inside** the block are **private**; the `parallel for` loop variable is private.

Portability: `#ifdef _OPENMP` can be used to check that the compiler supports OpenMP.

Q. 3 Explain different types of scheduling in OpenMP. [10 Marks]

Introduction

In OpenMP, **assigning loop iterations to threads is called scheduling**. It is controlled by the `schedule` **clause** of `parallel for` or `for`. The default is system dependent, usually a **block partition** (first `n/thread_count` iterations to thread 0, next to thread 1, ...). If the cost of iteration `i` grows with `i`, the last thread gets far more work; a **cyclic partition** (round-robin) balances better.

Textbook result (`n = 10,000`): 1 thread = 3.67 s; 2 threads default = 2.76 s (speedup 1.33); 2 threads cyclic = 1.84 s (speedup **1.99**).

Syntax

```
schedule(<type> [, <chunksize>])

#  pragma omp parallel for num_threads(thread_count) \
       reduction(+: sum) schedule(static, 1)
   for (i = 0; i <= n; i++)
       sum += f(i);
```

A **chunk** is a block of consecutive iterations of the serial loop; `chunksize` is its size. Only `static`, `dynamic`, `guided` accept a chunksize.

Types of schedule

1. static

- Iterations are assigned to threads **before the loop executes**, in **round-robin** chunks of `chunksize`.
- If `chunksize` is omitted it is about `total_iterations / thread_count` (usual default).
- **Best when** all iterations take about the same time. Same ranges in later loops help NUMA memory access. Lowest overhead.

12 iterations, 3 threads	Thread 0	Thread 1	Thread 2
`static, 1`	0, 3, 6, 9	1, 4, 7, 10	2, 5, 8, 11
`static, 2`	0, 1, 6, 7	2, 3, 8, 9	4, 5, 10, 11
`static, 4`	0, 1, 2, 3	4, 5, 6, 7	8, 9, 10, 11

2. dynamic

- Iterations are broken into chunks. When a thread finishes a chunk, it **requests another from the run-time system**, until all are done. Default chunksize = **1**.
- **First-come, first-served**; useful when iterations take **non-uniform time**.
- More **overhead** than static (assigned at run time). Larger chunk = fewer assignments.

3. guided

- Similar to dynamic, but the **chunk size decreases** as chunks are completed: about (remaining iterations) / (number of threads).
- Example (`n = 10,000`, 2 threads): chunks 5000, 2500, 1250, 625, ... down to 1.
- With a `chunksize` they decrease down to `chunksize` (last chunk may be smaller). Good when **later iterations are costlier**. Highest overhead.

4. auto and 5. runtime

auto: the compiler and/or run-time system decides the schedule. **runtime:** the schedule is decided **at run time** from the environment variable `OMP_SCHEDULE`, e.g. `$ export OMP_SCHEDULE="static,1"`. Useful for testing many schedules without recompiling.

Which schedule?

- Scheduling adds overhead (dynamic > static; guided is the most). If the default is satisfactory, use no `schedule` clause.
- Iterations of equal cost: **default** is likely best.
- Cost increases or decreases linearly: **static with small chunksize**.
- Cost not known in advance: try several using `schedule(runtime)`.

Conclusion

A good schedule balances the load among threads and can greatly improve speedup.

Q. 4 Explain parallel loops using the parallel for directive. Write a coded example. [10 Marks]

Introduction

OpenMP can parallelize a serial `for` loop with small source changes. Place the `parallel for` directive immediately before the loop.

Working

- Like `parallel`, it **forks a team of threads** to execute the structured block, which **must be a `for` loop**.
- The system **divides the iterations** among the threads (default: usually block partition; changeable by `schedule`).
- The **loop variable is private** by default; other variables declared before the loop are **shared**.
- There is an **implicit barrier** at the end of the loop.

```
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
#  pragma omp parallel for num_threads(thread_count) reduction(+: approx)
for (i = 1; i <= n-1; i++)
    approx += f(a + i*h);
approx = h*approx;
```

`approx` must be a **reduction** variable; otherwise `approx += ...` is an unprotected critical section.

Caveats

1. Only `for` **loops** are parallelized (not `while` / `do-while`).
2. The **number of iterations must be known** before the loop starts. `for(;;)` and loops with `break` cannot be parallelized (gcc: "invalid exit from OpenMP structured block").
3. The loop must be in **canonical form**: `for (index = start; index < end; index++)` with test `<, <=, >=, >` and increment `index++`, `++index`, `index--`, `--index`, `index += incr`, `index -= incr`, `index = index + incr`, `index = incr + index`, `index = index - incr`.
4. `index` must be **integer or pointer** type (not `float`); `start`, `end`, `incr` must not change during the loop; `index` is modified only by the increment expression.

Data dependences

The compiler does **not** check for dependences. A **loop-carried dependence** (an iteration uses results of another) cannot normally be parallelized. Example (wrong): `fibo[i] = fibo[i-1] + fibo[i-2];` can print `1 1 2 3 5 8 0 0 0 0`. Only variables that are **updated** in the loop and read/written in another iteration matter.

Coded example: estimating pi (series 4[1 - 1/3 + 1/5 - 1/7 + ...])

The serial `factor = -factor` creates a loop-carried dependence. It is removed by computing `factor` from `k`. `factor` is then made **private**; `sum` is a **reduction** variable.

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main(int argc, char* argv[]) {
    int    thread_count = strtol(argv[1], NULL, 10);
    int    n = 1000, k;
    double factor, sum = 0.0, pi_approx;

#   pragma omp parallel for num_threads(thread_count) \
        default(none) reduction(+: sum) private(k, factor) shared(n)
    for (k = 0; k < n; k++) {
        factor = (k % 2 == 0) ? 1.0 : -1.0;     /* no loop-carried dependence */
        sum += factor / (2*k + 1);
    }
    pi_approx = 4.0 * sum;
    printf("With n = %d terms and %d threads,\n", n, thread_count);
    printf("   Our estimate of pi = %.14f\n", pi_approx);
    return 0;
}
```

Clauses used

- `num_threads(thread_count)`: number of threads.
- `reduction(+: sum)`: private `sum` per thread (initialised to 0), added into the shared `sum` at the end.
- `private(k, factor)`: own copy per thread; a shared `factor` would be overwritten by other threads and give a wrong result.
- `shared(n)`: only read, never updated. `default(none)`: forces scope of every variable to be stated.

Conclusion

`parallel for` parallelizes loops easily, provided the loop is in canonical form, loop-carried dependences are removed and variable scopes (`private`, `shared`, `reduction`) are set correctly.